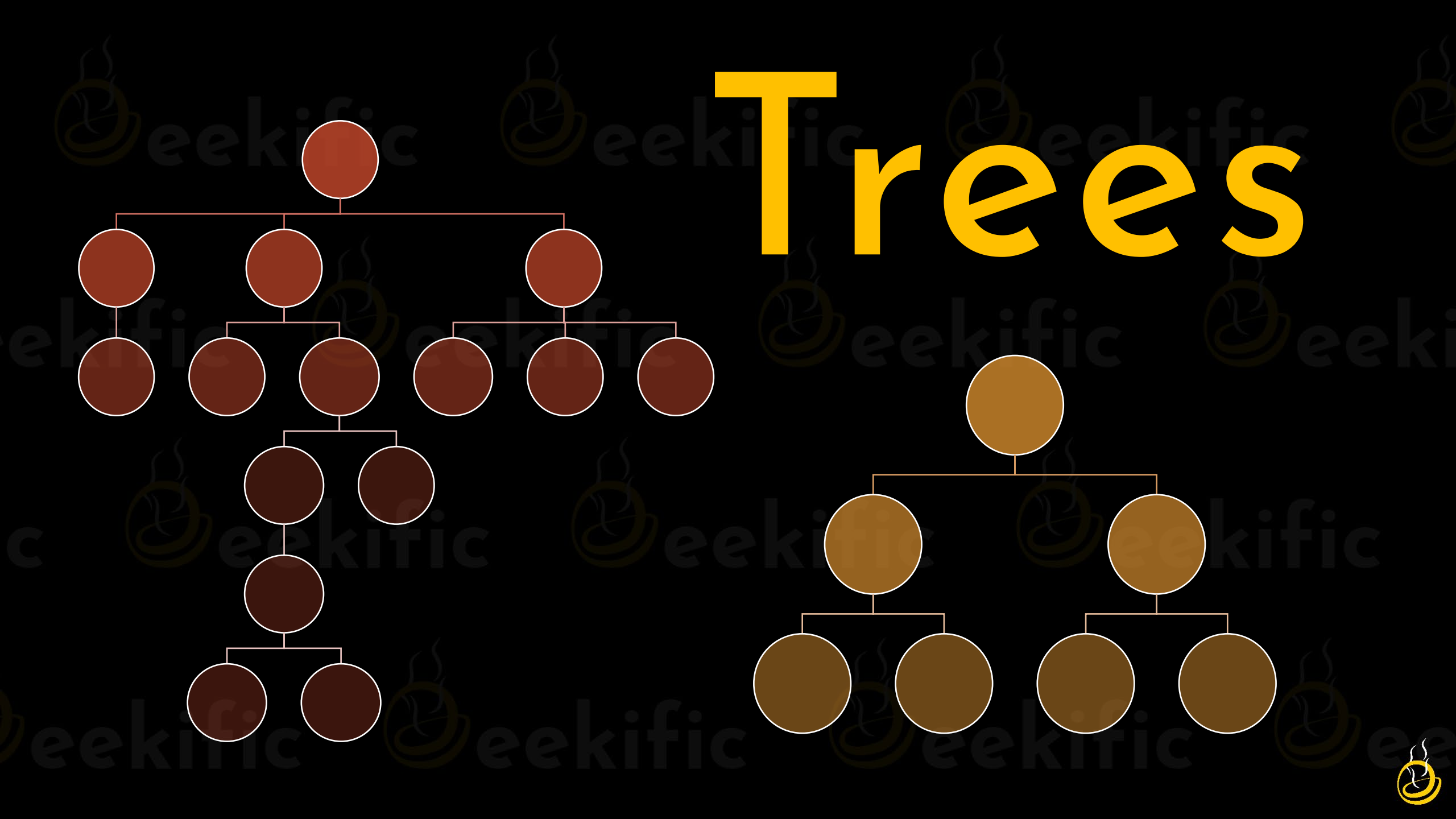
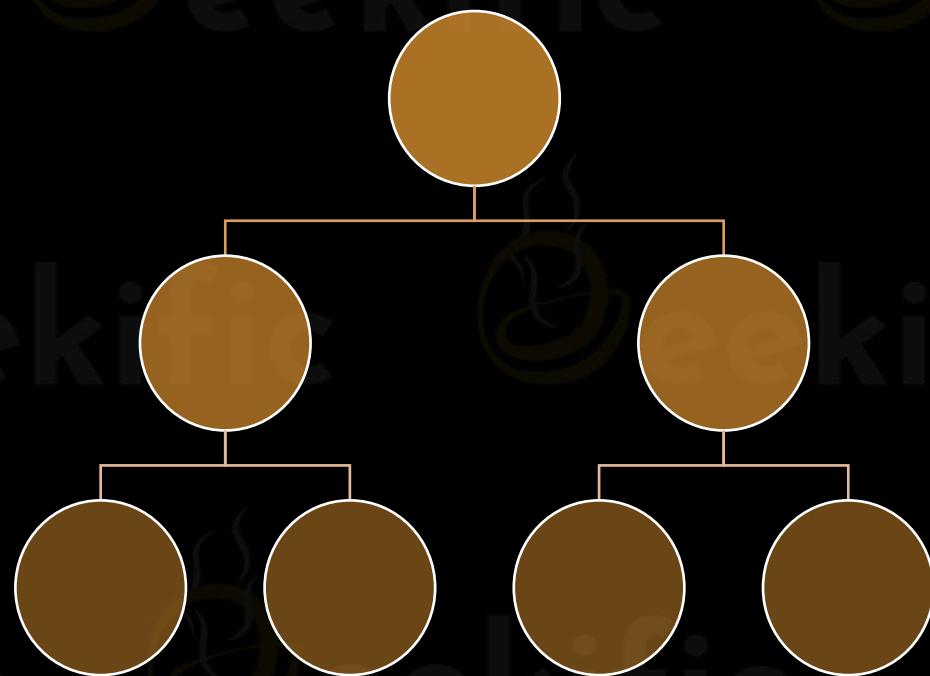
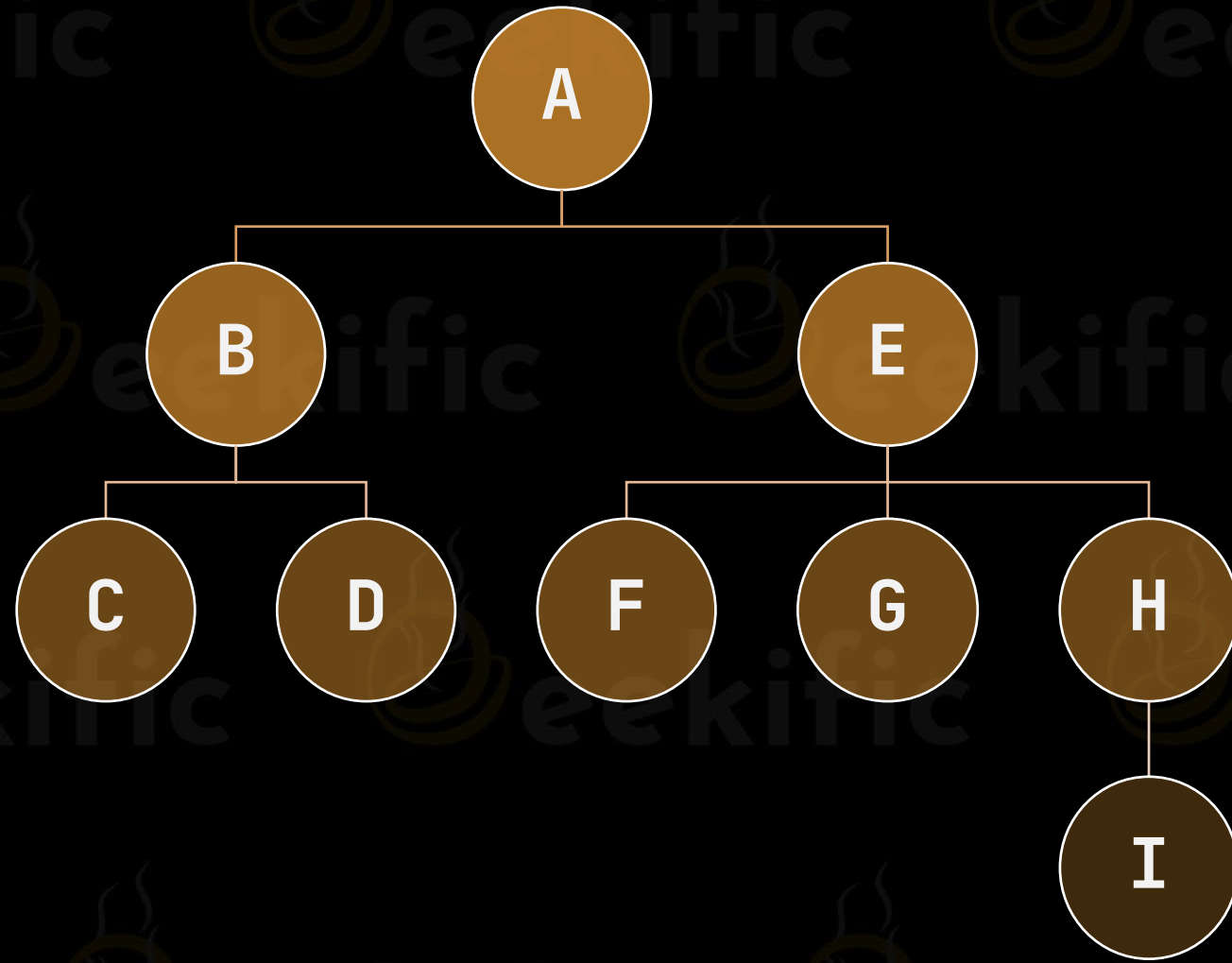


```

graph TD
    A(( )) --- B(( ))
    A --- C(( ))
    A --- D(( ))
    B --- E(( ))
    C --- F(( ))
    C --- G(( ))
    C --- H(( ))
    D --- I(( ))
    D --- J(( ))
    D --- K(( ))
    H --- L(( ))
    H --- M(( ))
    L --- N(( ))
    L --- O(( ))
  
```





# What is a Tree?

a **tree** is a collection of connected **nodes**

a **node** stores data and references to its **children**

the connections between nodes are called **edges**

if b is a child of a, then a is the **parent** of b

nodes sharing the same parent are **siblings**

a node with no children is a **leaf node**

the top node is the **root**

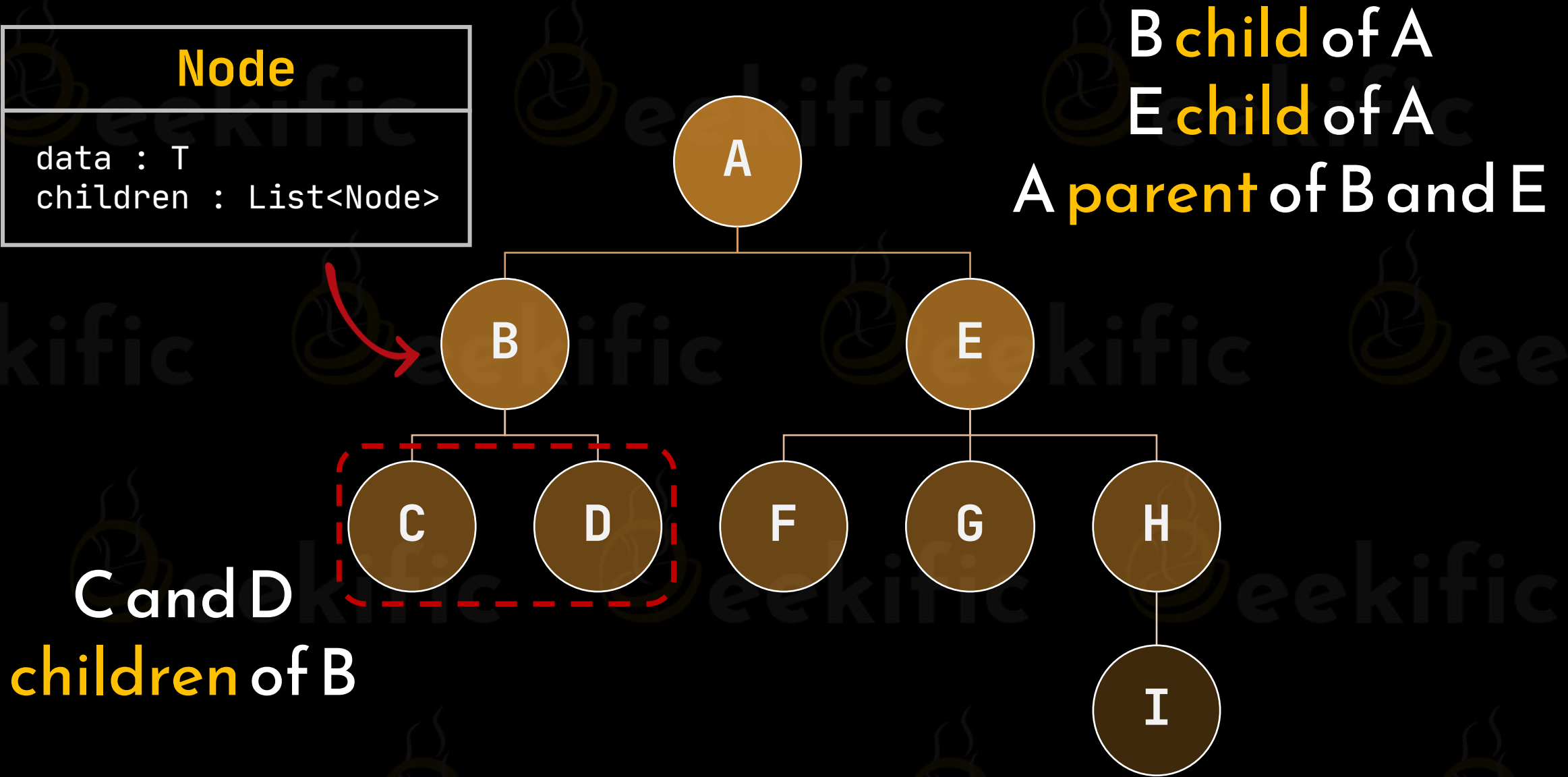
– like a linked list's head, it grants access to all other nodes



## Node

data : T

children : List<Node>



## Node

data : T

children : List<Node>



B child of A

E child of A

A parent of B and E

siblings

C and D  
children of B  
and siblings



| Node                              |
|-----------------------------------|
| data : T<br>children : List<Node> |

root node



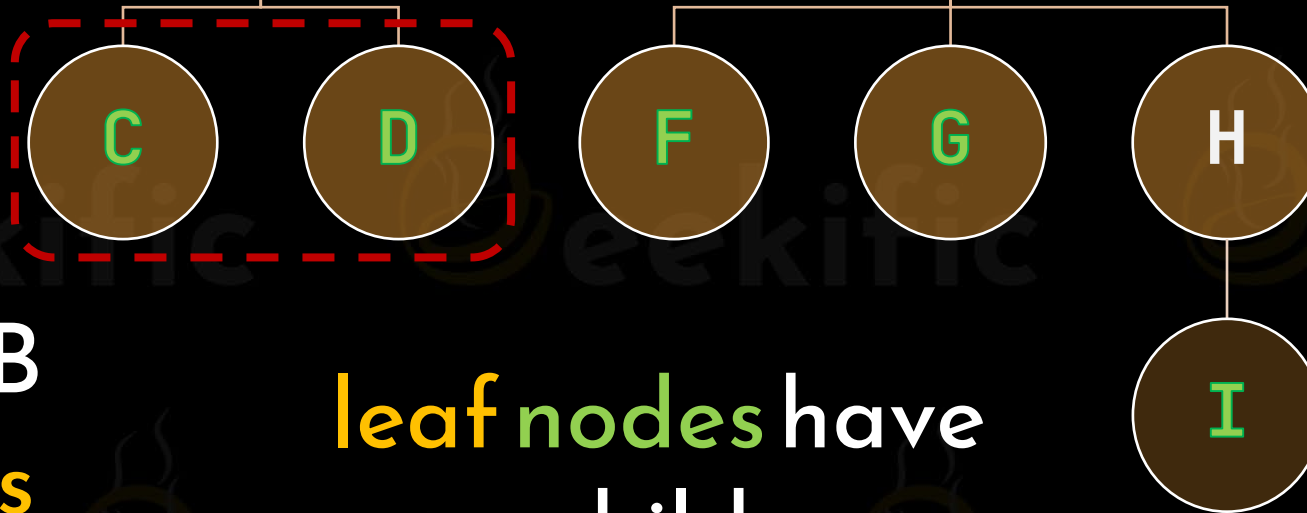
B child of A

E child of A

A parent of B and E



siblings



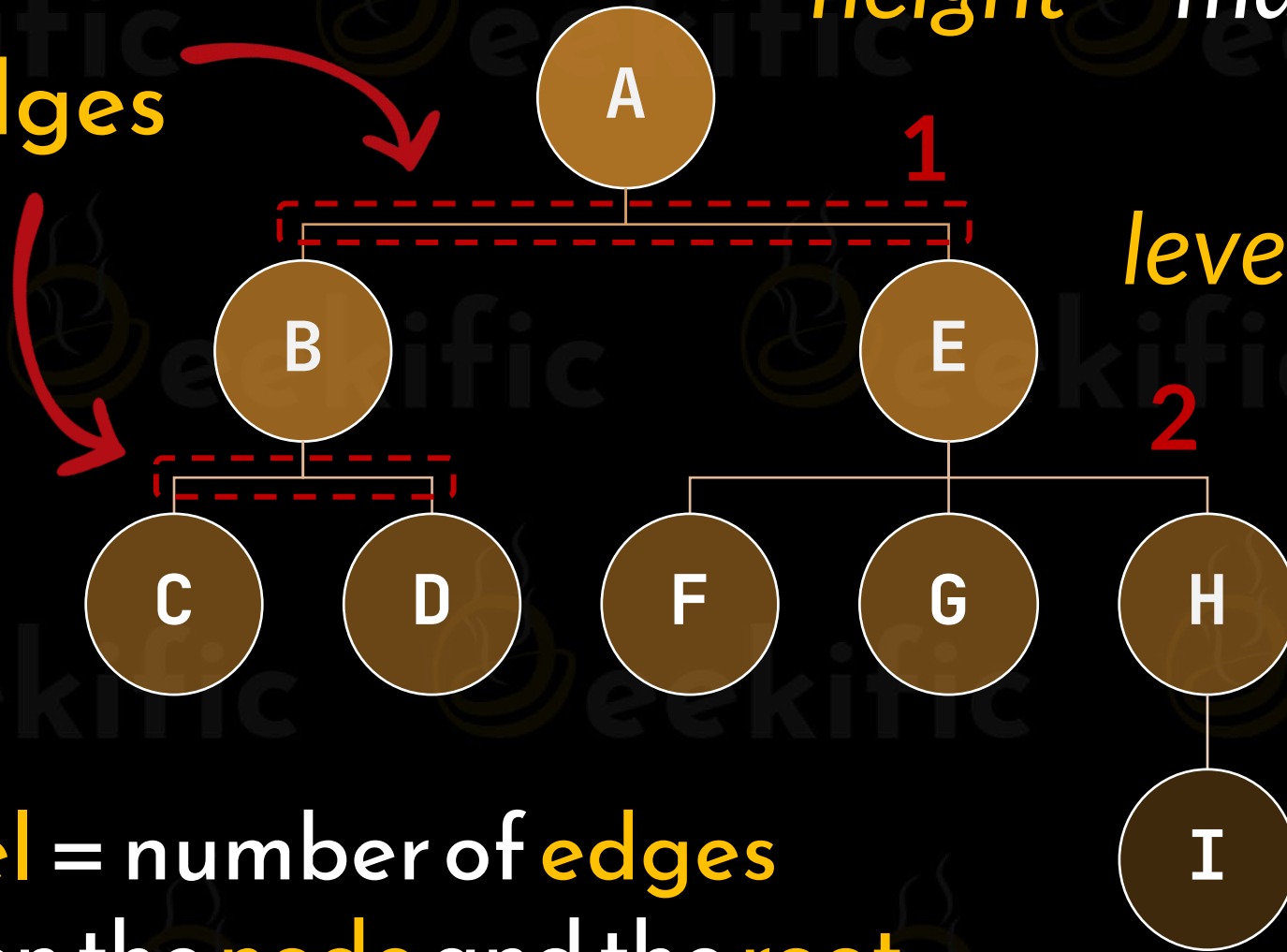
C and D  
children of B  
and siblings

leaf nodes have  
no children



$height = \max(level) = 3$

edges



$level\ of\ H = 2$

$level = \text{number of edges}$   
between the **node** and the **root**



# What Makes a Valid Tree?

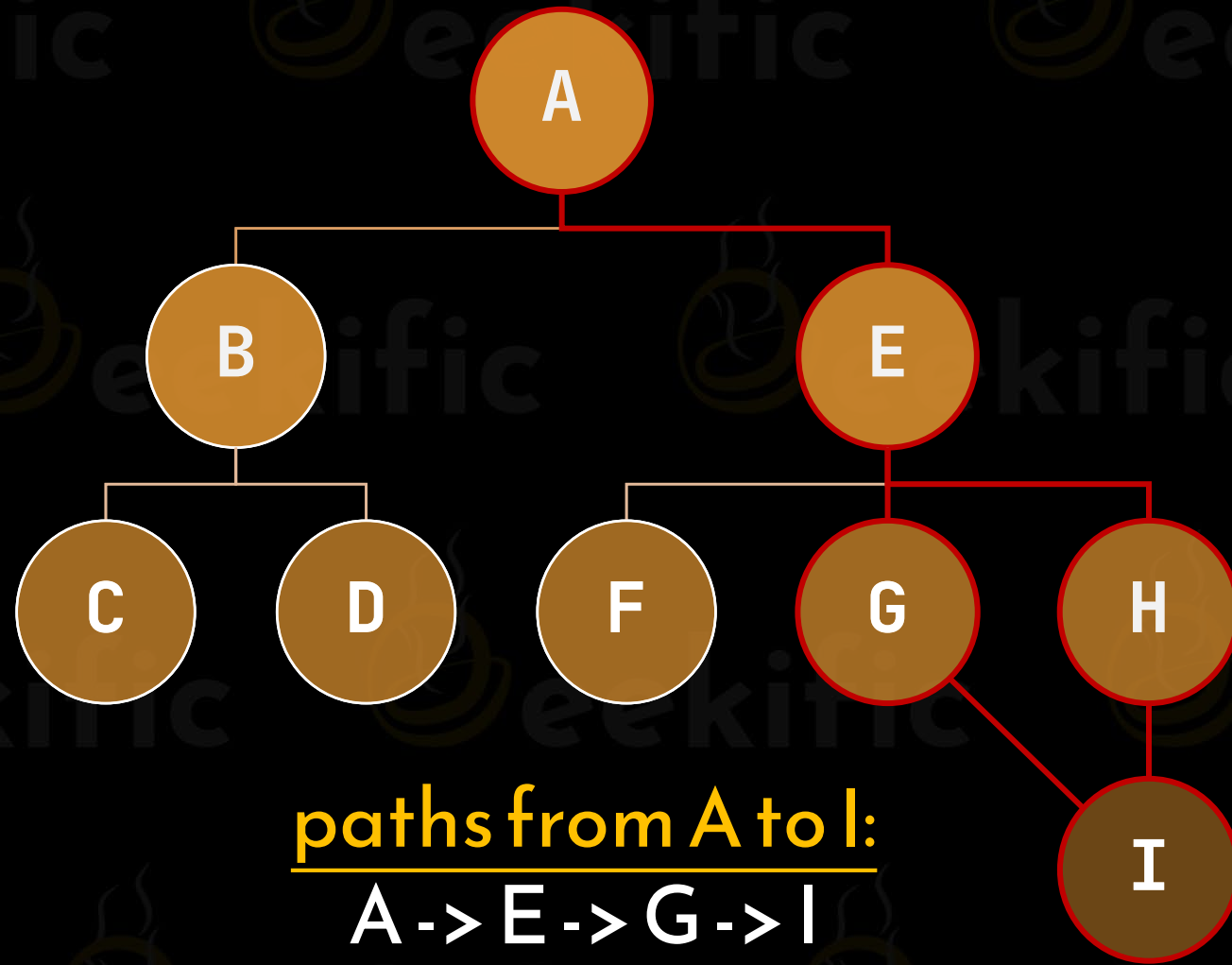
there must be a **single path** from the root to any other node in the tree

a tree must be **acyclic** – it cannot contain any cycles

for  **$n$  nodes**, there must be exactly  **$n - 1$  edges**, otherwise it is not a tree

a tree is actually a special type of **graph** – one that is undirected,  
connected, and acyclic





paths from A to I:

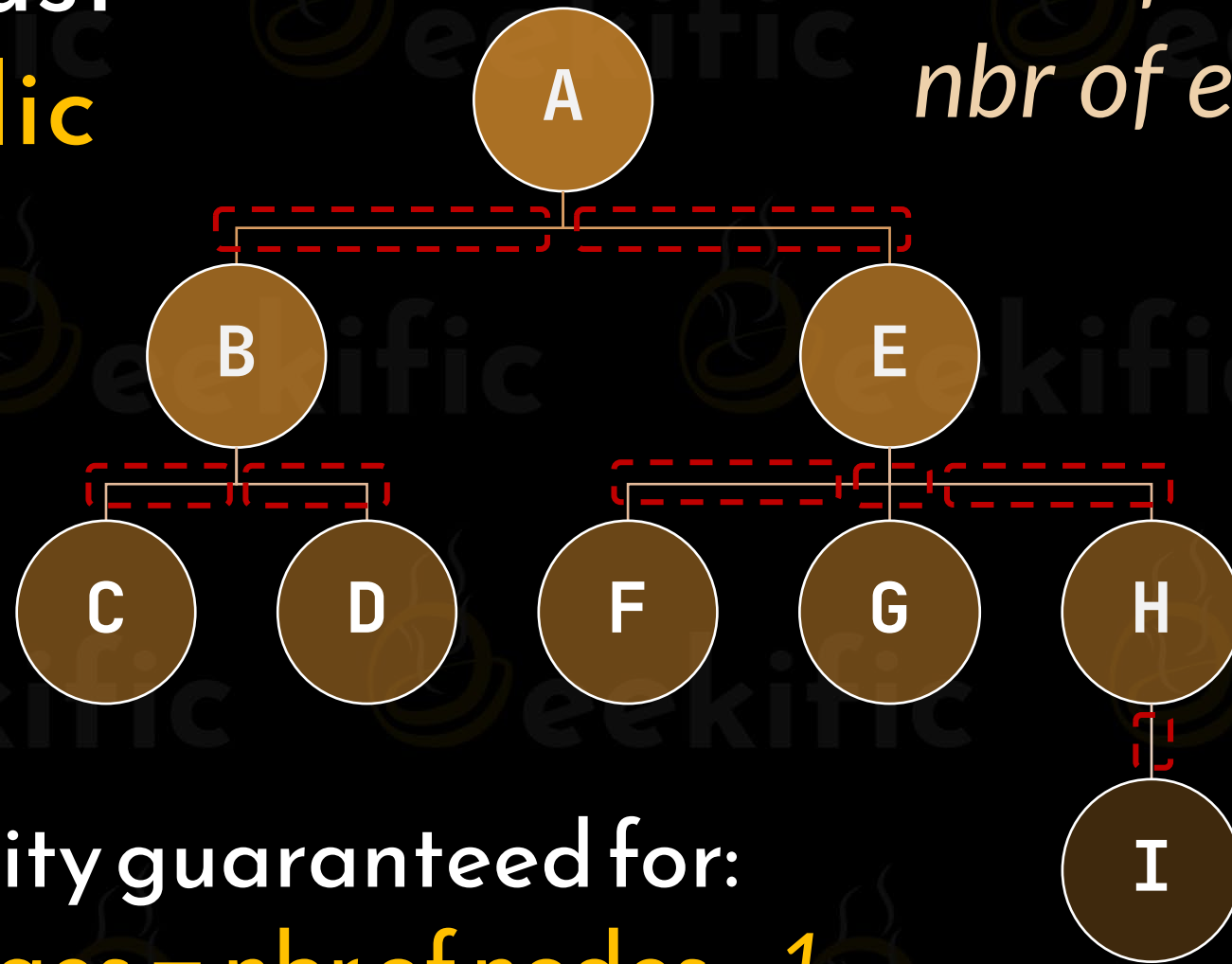
A -> E -> G -> I

A -> E -> H -> I



a tree must  
be **acyclic**

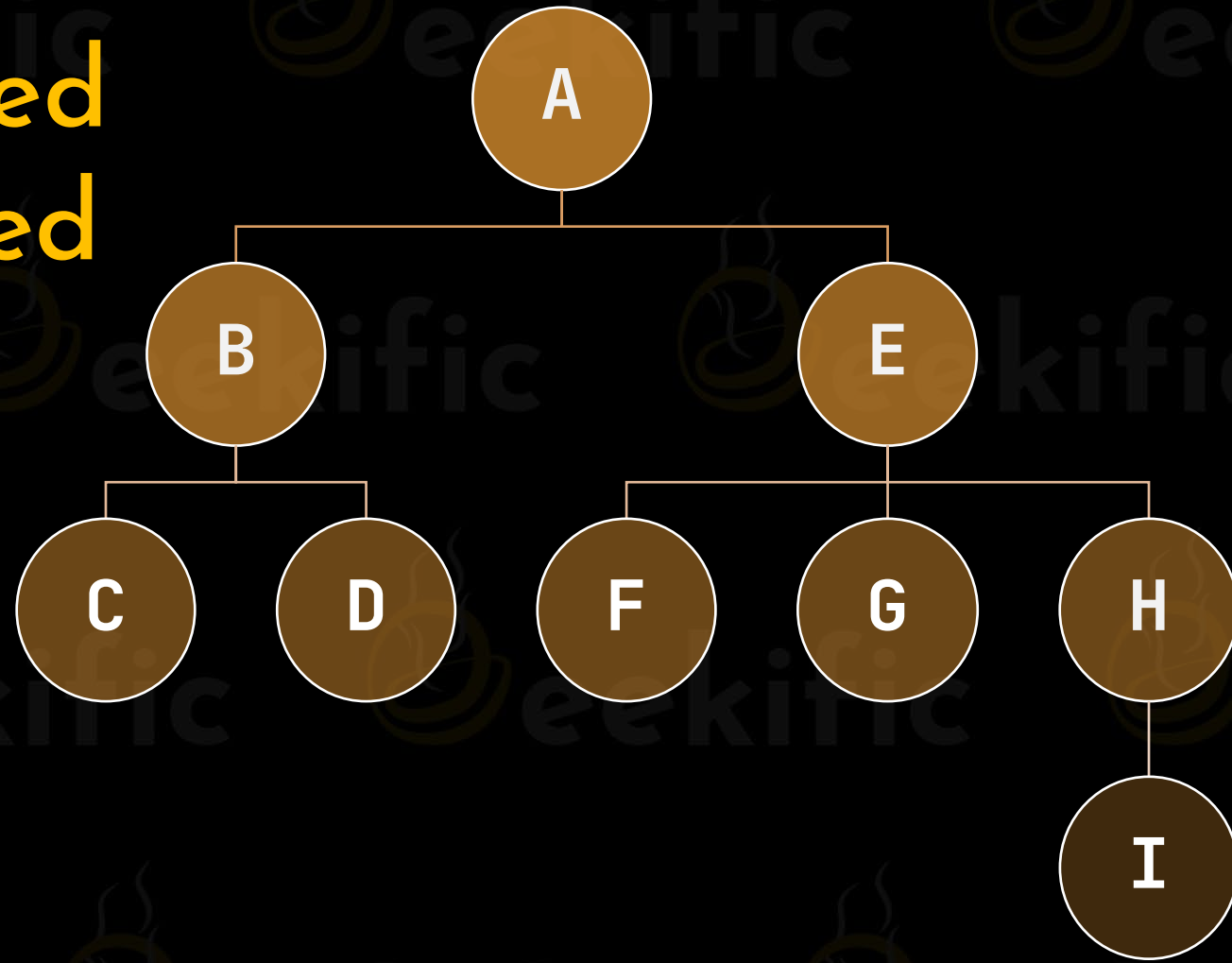
*nbr of nodes = 9*  
*nbr of edges = 8*



acyclicity guaranteed for:  
**nbr of edges = nbr of nodes - 1**



Tree:  
Undirected  
Connected  
Acyclic



# Tree Types

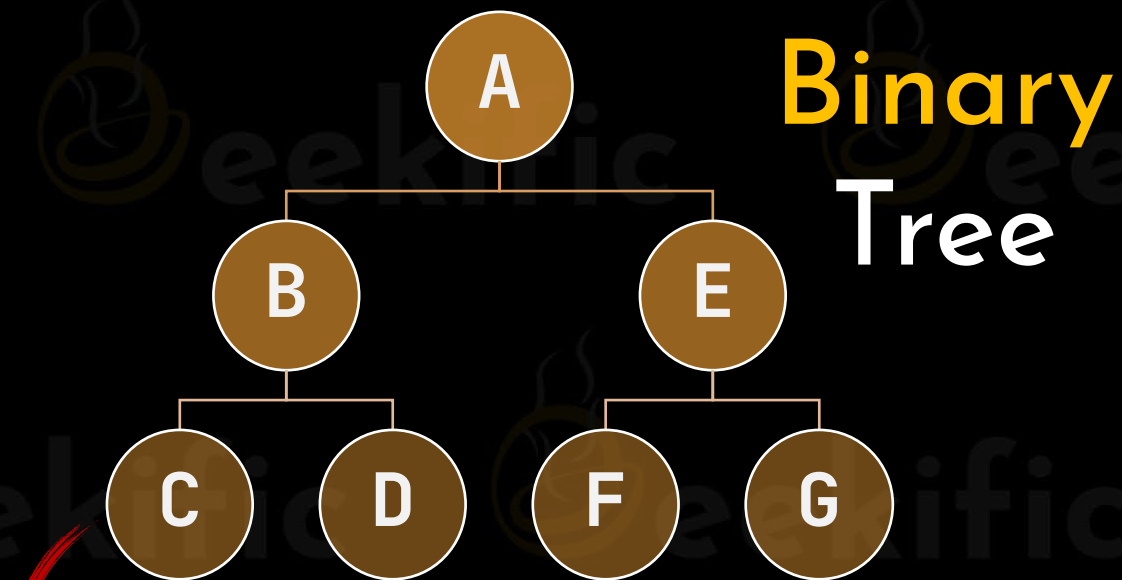
a **binary tree** allows each node to have at most **2 children**

a **ternary tree** allows each node to have at most **3 children**

a tree is **full** when every non-leaf node has the maximum number of children and all levels are filled

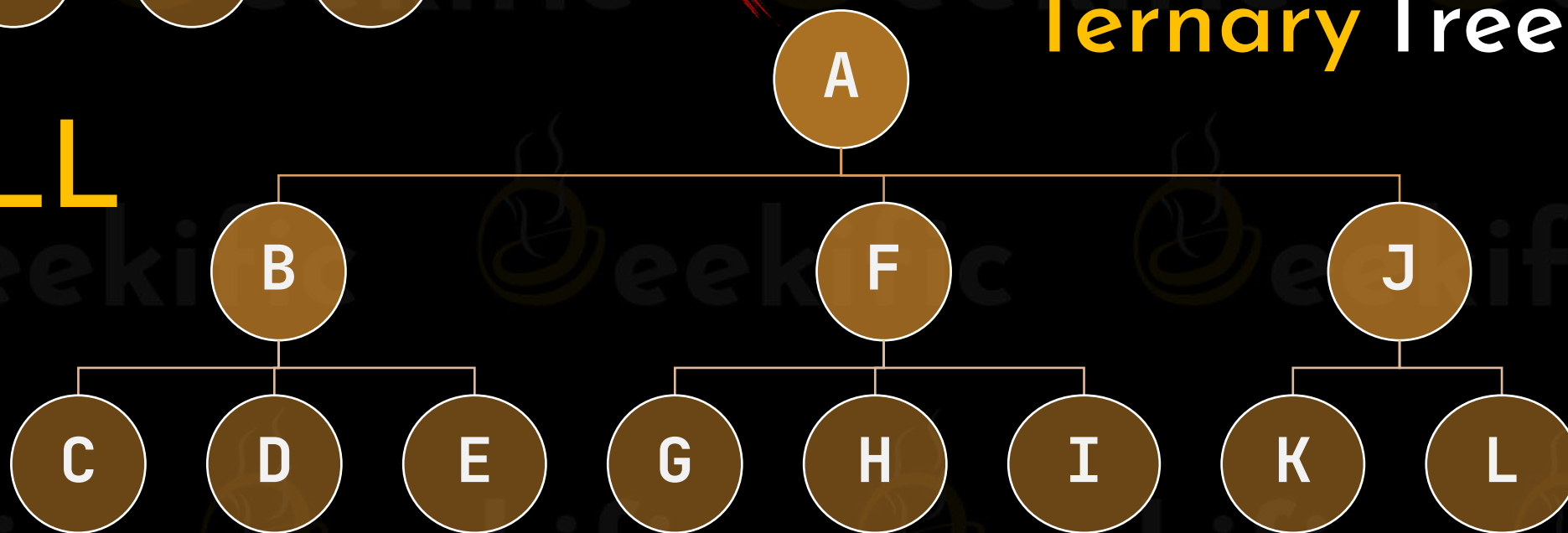
a tree is **complete** when every level except possibly the last is completely filled, with nodes as far left as possible





**FULL**

**COMPLETE**



# Binary Search Trees

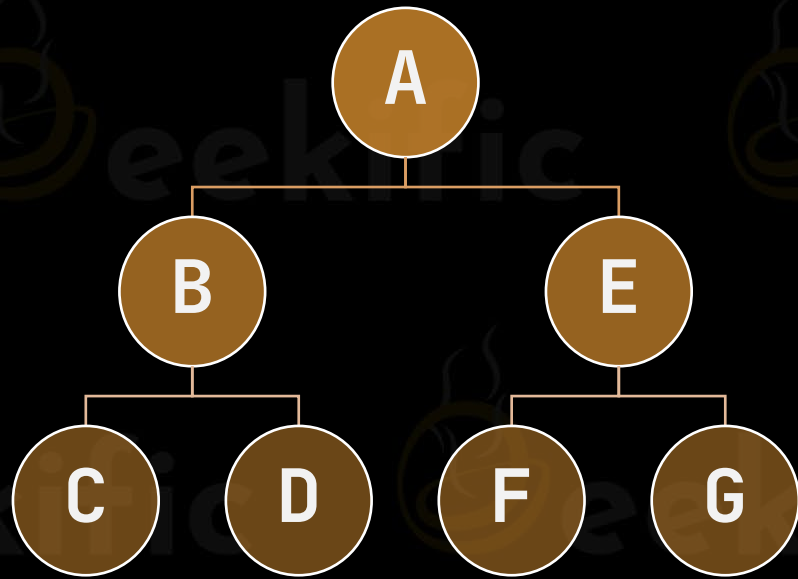
in a **bst**, the left subtree holds values **smaller** than the parent, and the right subtree holds values **greater**

this structure enables **fast lookup, addition, and removal** of data items

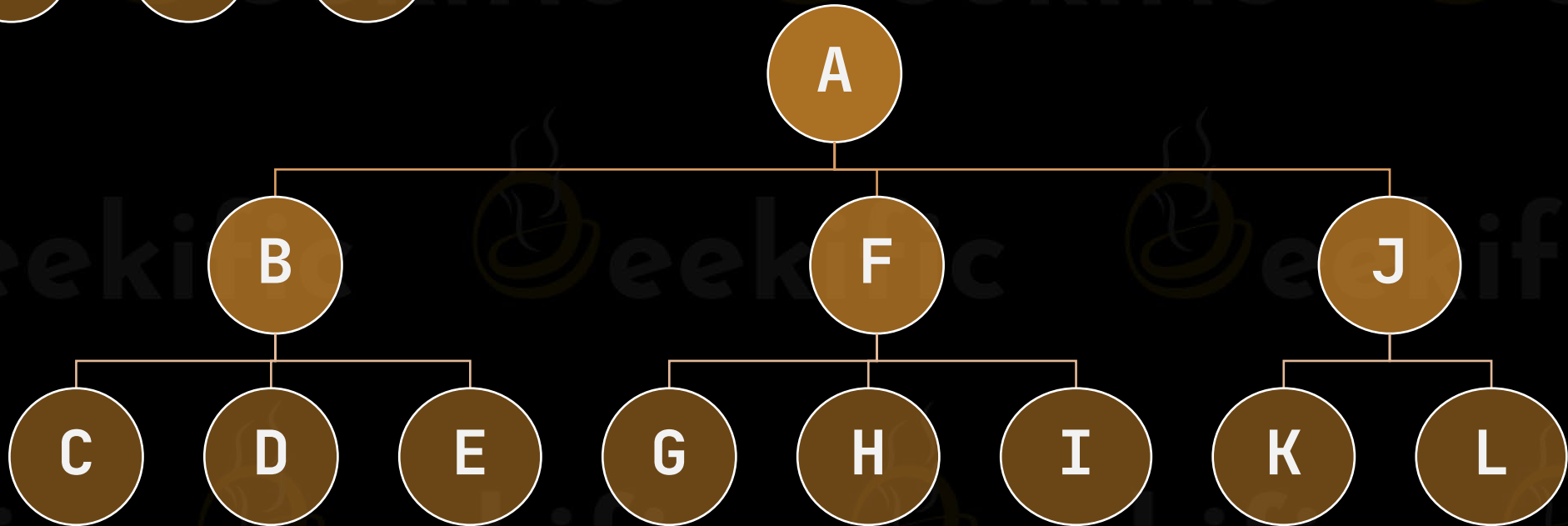
bsts can be used to implement **dynamic sets, lookup tables, and associative maps**

ternary search trees extend this concept, adding the ability for **incremental string search**



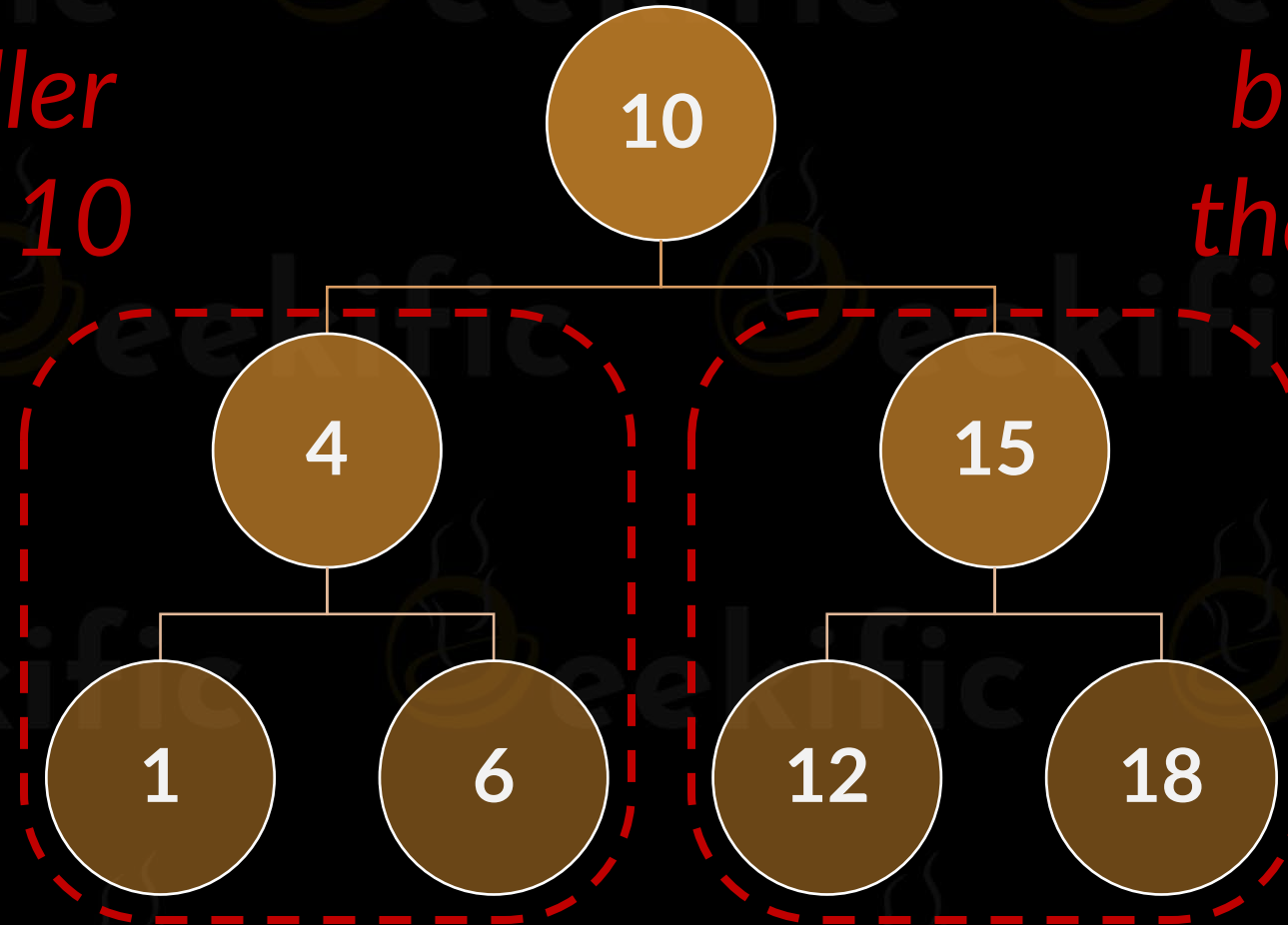


- ✓ Fast lookup
- ✓ Addition and removal of data items
- ✓ Implementation of lookup tables and associative maps



*smaller  
than 10*

*bigger  
than 10*



# The Balance Problem

a bst can degenerate into a straight line,  
essentially becoming a **list** – losing all tree advantages

a tree is **balanced** when the height difference between its left and right  
subtrees does not exceed **one**, given both subtrees are also balanced

to solve this, **AVL trees** and **Red-Black trees** were created  
– they automatically maintain balance after every operation



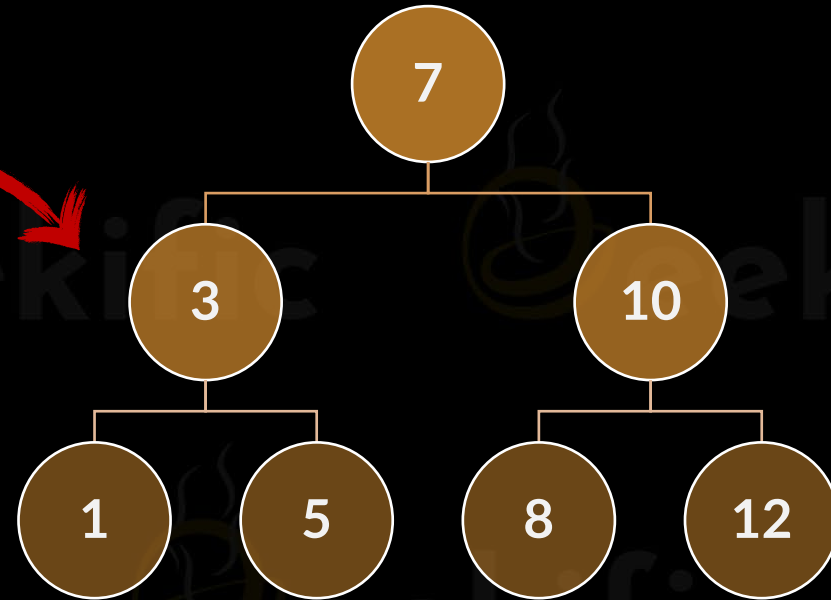


to solve this:  
AVL and Red-Black Trees  
were born



self-balancing trees

## AVL and Red-Black Trees



# Heaps

a heap is another implementation of binary trees,  
with two variants: **maximum** and **minimum**

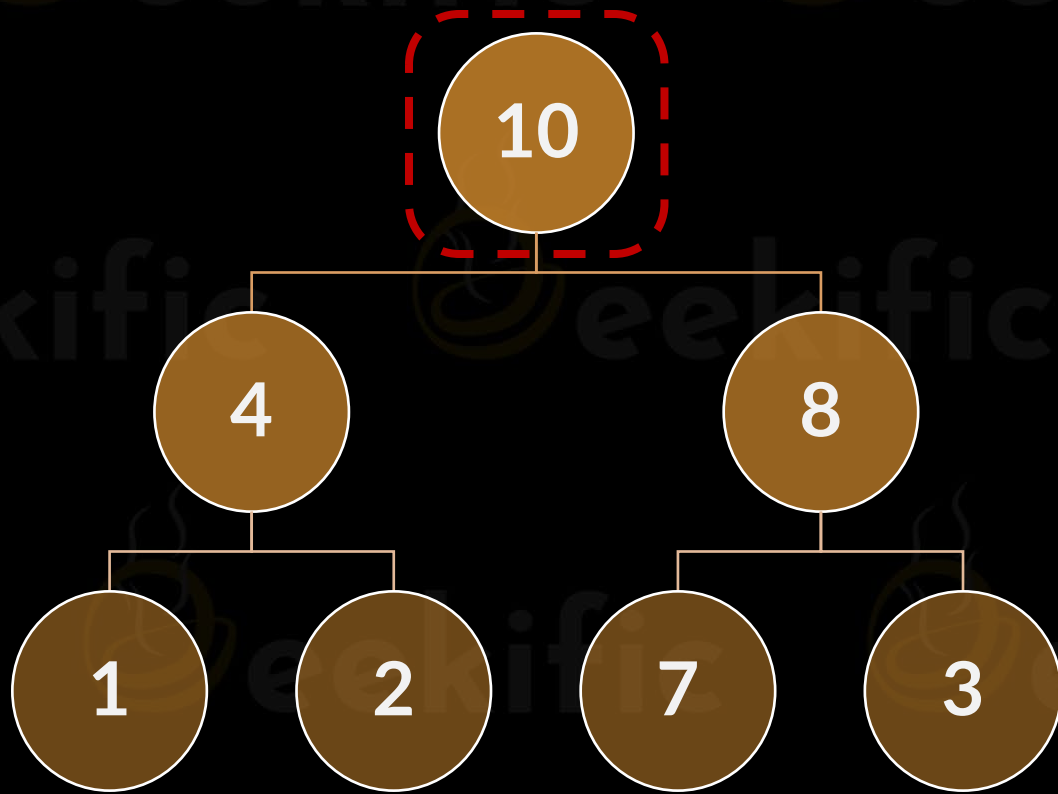
in a **max heap**, parent nodes are always **bigger**  
than their children – the root stores the largest value

in a **min heap**, parent nodes are always **smaller**  
than their children – the root stores the smallest value

unlike bsts, heaps do not care whether the  
left child is smaller or larger than the right child

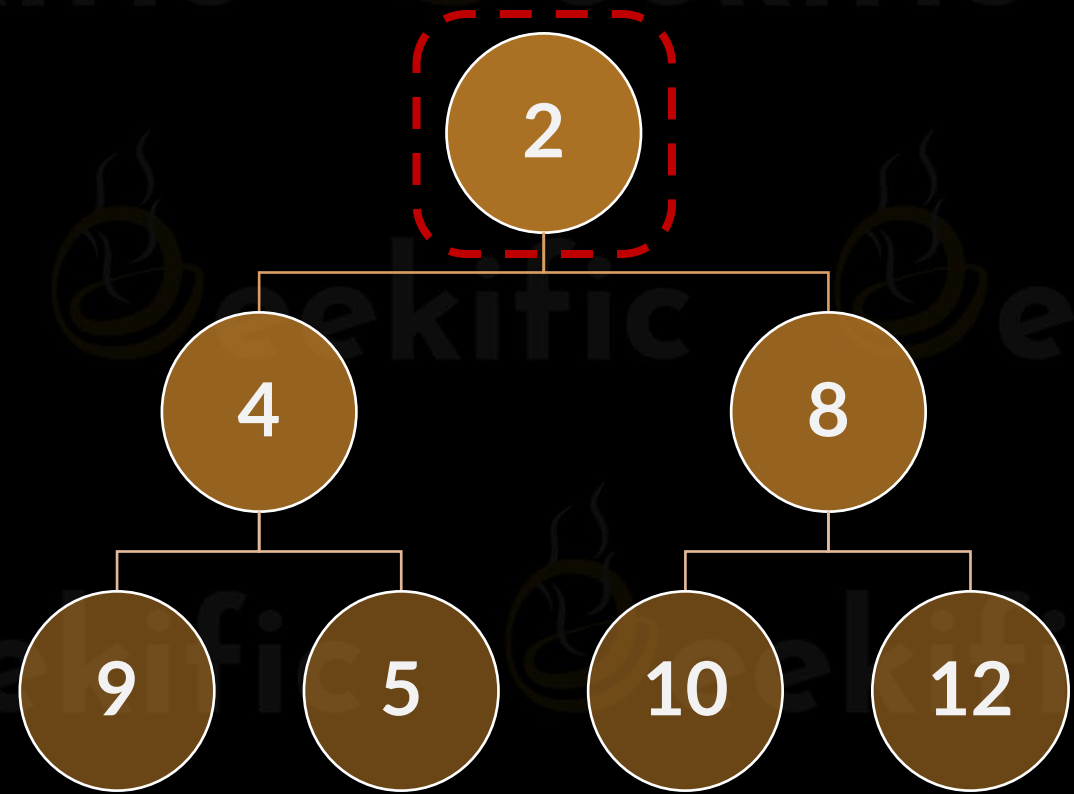


*biggest node*



**Maximum Heap**

*smallest node*



**Minimum Heap**



# Tries

a **trie** (prefix tree) is a search tree  
used to store **strings** character by character

applications include **text prediction**,  
**auto-completion**, and **lexicographic sorting**

a **suffix tree** indexes all suffixes in a text,  
enabling fast **full text search**

ternary trees are a type of trie,  
efficient for prefix-based lookups





# Trie

text prediction

auto-completion

lexicographic

sorting



# Key Takeaways

A **tree** is an acyclic graph of connected nodes, accessed through its **root**

**Binary search trees** enable fast operations by enforcing **left < parent < right** ordering

**AVL** and **Red-Black trees** solve degeneration through **self-balancing**

**Heaps** prioritize parent-child ordering (max or min) without left-right constraints

**Tries** store strings for prefix-based search; **suffix trees** enable full text indexing

